FIELD NOTES

Slack Field Guide

Slack answers almost everything with HTTP 200 and puts the failure in the body as ok: false, so a bot that never posted looks like one that did. Every script here is read only.

52 fixes, each one a written note with diagrams and a small Python and Node.js script that finds the problem and repairs it. All 52 have a script in the public repo.

Before you run anything. Every script starts in a dry run: it reports what it would change and writes nothing. Read that output first, then set `DRY_RUN=false` when you are satisfied it has understood your data.

Scripts: github.com/allanninal/slack-fixes
Guides: allanninal.dev/slack
All 14 repos: github.com/allanninal

The fixes

1 **accesslimited: the right token from the wrong network**

The token works. Someone pastes the same curl into their laptop terminal and it comes back ok: true, in front of witnesses. The identical command in the production container answers {"ok": false, "error": "accesslimited"}, and the deploy that shipped an hour earlier gets blamed for it.

`slack_egress_allowlist.py`

`https://www.allanninal.dev/slack/accesslimited-ip-allowlist/`

`https://github.com/allanninal/slack-fixes/tree/main/accesslimited-ip-allowlist`

---

2 **account_inactive: the installer left and took the token**

The nightly export ran for two years and stopped on a Tuesday. Nothing was deployed, no scope changed, the app is still listed in the workspace. The error is {"ok": false, "error": "account_inactive"}, and the explanation is in the HR system rather than yours: the engineer who installed the app in 2024 left the company on Monday, and SSO deprovisioning deactivated her account overnight.

`slack_installer_account_watch.py`

`https://www.allanninal.dev/slack/account-inactive/`

`https://github.com/allanninal/slack-fixes/tree/main/account-inactive`

---

3 **Socket Mode never connects: the xapp- token is underscoped**

The process starts, Bolt announces that it is connecting, and then it retries. Forever. No events arrive, nothing crashes, and the only line in the log that matters is missing_scope from apps.connections.open — a method your read-only audit script is not allowed to call, because opening a connection is a write.

`slack_socket_readiness.py`

https://www.allanninal.dev/slack/app-level-token-missing-connections-write/

https://github.com/allanninal/slack-fixes/tree/main/app-level-token-missing-connections-write

---

4 **is_archived: the target channel was archived months ago**

Nobody filed a ticket, because nothing looks broken. The channel is still there if you search for it, the ID in the config still resolves, and conversations.info still answers ok: true. What changed is one boolean inside that response, set during a reorganisation in March, and every alert since has been refused on the way in.

`slack_archived_target_sweep.py`

https://www.allanninal.dev/slack/archived-channel-target/

https://github.com/allanninal/slack-fixes/tree/main/archived-channel-target

---

5 **One event, many installs: authorizations:read fans it out**

Support says the bot only works for some customers. Your logs disagree: every event was received, handled, and acknowledged, with no errors on any of them. Both are true. Slack delivered one event that several installations should have seen, your handler processed it for the one workspace named in the payload, and nothing anywhere recorded the ones it skipped.

`slack_event_fanout_audit.py`

https://www.allanninal.dev/slack/authorizations-read-missing/

https://github.com/allanninal/slack-fixes/tree/main/authorizations-read-missing

---

6 **The bot joined 4,000 channels and the nightly sweep dies**

The job that enumerates the bot's channels used to take four seconds. It now takes eleven minutes, and last Tuesday it stopped taking any time at all because it died on ratelimited before it finished. Nothing in the code changed. Nothing is misconfigured. The bot is a member of every channel it is a member of, legitimately, because an onboarding automation has been inviting it to every new channel for two years.

`slack_channel_footprint.py`

https://www.allanninal.dev/slack/bot-in-too-many-channels/

https://github.com/allanninal/slack-fixes/tree/main/bot-in-too-many-channels

---

7 **the bot answers its own messages in an endless loop**

A channel fills with hundreds of identical bot messages in a few seconds. Slack starts rate-limiting the app, which slows the flood without stopping it, and in the end somebody removes the bot from the channel to make it stop. The cause is one line that was never written: the handler subscribed to message.channels, which delivers every message in the channel, including the one the app posted a moment ago.

`slack_echo_loop_audit.py`

https://www.allanninal.dev/slack/bot-message-echo-loop/

https://github.com/allanninal/slack-fixes/tree/main/bot-message-echo-loop

---

8 **not_in_channel: the bot was never invited to the channel**

The app is installed. The token authenticates. The channel ID was copied out of the URL and is correct. Every call still comes back {"ok": false, "error": "not_in_channel"}, because installing an app to a workspace does not put it in a single channel — and this is, by view count, the most-asked Slack API question there is.

`slack_channel_membership.py`

https://www.allanninal.dev/slack/bot-not-in-channel/

https://github.com/allanninal/slack-fixes/tree/main/bot-not-in-channel

---

9 **the scope was granted to the user token, not the bot**

You added the scope. The consent screen listed it, an admin approved it, you reinstalled, you replaced the stored token, and the call still returns missing_scope with that exact scope in needed. Open the app configuration and there it is, plainly granted — under User Token Scopes, while every line of your code authenticates with the xoxb- bot token.

`slack_token_identity_split.py`

https://www.allanninal.dev/slack/bot-vs-user-scope-mixup/

https://github.com/allanninal/slack-fixes/tree/main/bot-vs-user-scope-mixup

---

10 **is_private: the public channel you read was converted**

The reader has been pulling history out of #incidents for a year. This morning it returns channel_not_found, and the day before it returned messages. Nothing was deployed, the ID in the config is the ID it has always been, and the channel is still there. Somebody converted it to private on Tuesday, which moved it across a scope boundary without moving its identifier.

`slack_visibility_change.py`

https://www.allanninal.dev/slack/channel-converted-to-private/

https://github.com/allanninal/slack-fixes/tree/main/channel-converted-to-private

---

11 **channel_not_found: a channel name where an ID belongs**

{"ok": false, "error": "channel_not_found"} for a channel that is open on the other monitor. The name in the config is spelled right, the bot is in the room, the token is fine. And the daily digest that posts to that same string has been working for a year. The difference is not the channel. It is which method you handed the string to.

`slack_channel_reference_form.py`

https://www.allanninal.dev/slack/channel-name-instead-of-id/

https://github.com/allanninal/slack-fixes/tree/main/channel-name-instead-of-id

---

12 **channel_not_found after a rename, or worse, the wrong room**

An integration that has run untouched for two years stops on a Tuesday. Nothing was deployed, no token expired, no scope changed. What happened is that a team tidied up and renamed #ops-alerts to #platform-alerts. If you are lucky, you get channel_not_found and a page. If you are not, somebody created a new #ops-alerts the following week and your alerts are still being delivered, to strangers.

`slack_channel_name_drift.py`

https://www.allanninal.dev/slack/channel-renamed-hardcoded/

https://github.com/allanninal/slack-fixes/tree/main/channel-renamed-hardcoded

---

13 **Classic Slack app: the scope list says bot, client, read**

You go to add one scope. The OAuth & Permissions page does not offer the scope picker you have seen in every tutorial, the code references chat:write:bot which the documentation says does not exist, and the header on every API response reads bot,client,identify,post,read. Nothing is broken yet. What is broken is the assumption that this app can be fixed by adding something to it.

`slack_classic_scope_audit.py`

https://www.allanninal.dev/slack/classic-app-coarse-scopes/

https://github.com/allanninal/slack-fixes/tree/main/classic-app-coarse-scopes

---

14 **The Slack app configuration token died twelve hours ago**

The manifest step in CI passed yesterday and fails today. Someone regenerates the token by hand, the build goes green, and the same thing happens tomorrow morning. Meanwhile the audit that runs beside it reports every manifest-derived check as clean, because a check that could not run returned nothing to complain about.

`slack_config_token_state.py`

https://www.allanninal.dev/slack/config-token-expired/

https://github.com/allanninal/slack-fixes/tree/main/config-token-expired

---

15 **channel_not_found: the DM conversation was never opened**

The onboarding bot DMs every new hire. It works for the twelve people who tried it during the pilot and fails for everybody since with channel_not_found, for a user ID copied straight out of the profile panel. Nothing is wrong with the ID. There is no conversation for it to be delivered into, because a direct message is a channel and nobody has ever opened this one.

`slack_dm_targets.py`

https://www.allanninal.dev/slack/dm-never-opened/

https://github.com/allanninal/slack-fixes/tree/main/dm-never-opened

---

16 **DMs delivered into deactivated accounts, with ok: true**

The approval bot has a 96% delivery rate and a 41% response rate, and nobody can explain the gap. Every send returns ok: true. The messages are arriving, in DMs, with people who left the company: the account is deactivated, the conversation still exists, and Slack is faithfully writing into a room that nobody will ever open again.

`slack_dm_recipient_audit.py`

https://www.allanninal.dev/slack/dm-to-deactivated-user/

https://github.com/allanninal/slack-fixes/tree/main/dm-to-deactivated-user

---

17 **the same message posted three times, and the ts says why**

The alert channel has the same incident in it four times. Every one of those messages is a real, successful chat.postMessage call that returned ok: true and a distinct ts, so nothing failed and nothing will appear in your error tracker. The duplicates are not a display bug — they are four separate decisions your system made to send, and the record of all four is sitting in conversations.history waiting to be read.

`slack_duplicate_messages.py`

https://www.allanninal.dev/slack/duplicate-messages-no-dedupe/

https://github.com/allanninal/slack-fixes/tree/main/duplicate-messages-no-dedupe

---

18 **The dedupe key that does not survive the retry**

There is a dedupe check in the handler. Somebody added it after the last incident, it has a Redis set behind it and a TTL and a test, and there are still three Jira tickets for one Slack message.

`slack_dedupe_key_audit.py`

https://www.allanninal.dev/slack/duplicate-processing-on-retry/

https://github.com/allanninal/slack-fixes/tree/main/duplicate-processing-on-retry

---

19 **installs keyed on team_id alone collide on Enterprise Grid**

Two customers file the same ticket in the same week: messages from their Slack app are arriving in a channel that belongs to somebody else. Both are on the same Enterprise Grid organisation. Your installation store has one row per team_id, it has always had one row per team_id, and on a single-workspace customer that was correct. On Grid it is a cross-tenant data leak with a green dashboard.

`slack_install_key_audit.py`

https://www.allanninal.dev/slack/enterprise-id-not-stored/

https://github.com/allanninal/slack-fixes/tree/main/enterprise-id-not-stored

---

20 **A subscribed event whose scope the token never received**

The app handles messages in public channels perfectly. In private channels it is deaf. The subscription is there, on the same screen, one line below the one that works; the handler is the same function; the logs have nothing in them because there is nothing to log.

`slack_event_scope_gap.py`

https://www.allanninal.dev/slack/event-scope-mismatch/

https://github.com/allanninal/slack-fixes/tree/main/event-scope-mismatch

---

21 **Slack disabled event delivery and will not turn it back on**

There was a two-hour outage on Tuesday. The service came back, the health checks went green, the on-call went to bed. On Thursday somebody asks why the bot has not answered anyone since Tuesday. Slack turned event delivery off during the outage, emailed the app owner about it, and does not turn it back on when you recover — a human has to click a button that nobody knows exists.

`slack_event_silence_audit.py`

https://www.allanninal.dev/slack/event-subscriptions-auto-disabled/

https://github.com/allanninal/slack-fixes/tree/main/event-subscriptions-auto-disabled

---

22 **files.upload is retired: one probe returns method_deprecated**

Every internal tool that posts a screenshot into Slack stopped posting screenshots, all of them on the same day, none of them deployed that week. The logs say 200. The bodies say {"ok": false, "error": "method_deprecated"}. Nothing broke: files.upload reached the end of a sunset that was announced eighteen months earlier and moved once.

`slack_files_upload_probe.py`

https://www.allanninal.dev/slack/files-upload-retired/

https://github.com/allanninal/slack-fixes/tree/main/files-upload-retired

---

23 **posting_to_general_channel_denied: the default channel**

Every other channel takes the message. The one that does not is the channel the integration was pointed at on the afternoon somebody wired it up, because it was the channel that already existed and already had everyone in it. The webhook comes back HTTP 403 with three words of plain text. chat.postMessage comes back restricted_action. No scope you add will move either of them.

`slack_general_channel_target.py`

https://www.allanninal.dev/slack/general-channel-restricted/

https://github.com/allanninal/slack-fixes/tree/main/general-channel-restricted

---

24 **slack answers HTTP 200 and puts the failure in the body**

The deploy is green. The log line reads POST https://slack.com/api/chat.postMessage 200. Nothing has appeared in the channel for three weeks. When somebody finally logs the response body it reads {"ok": false, "error": "not_in_channel"} — and it has read that, unchanged, every single time.

`slack_ok_false_audit.py`

https://www.allanninal.dev/slack/http-200-ok-false/

https://github.com/allanninal/slack-fixes/tree/main/http-200-ok-false

---

25 **invalid_auth: the xapp- token is in the Web API slot**

{"ok": false, "error": "invalid_auth"} on a call that plainly should work, with a token copied out of the app configuration ten minutes ago. The token is not expired, not revoked, and not missing a scope. It starts with xapp-, and the Web API has never accepted one of those.

`slack_token_class_check.py`

https://www.allanninal.dev/slack/invalid-auth-wrong-token-type/

https://github.com/allanninal/slack-fixes/tree/main/invalid-auth-wrong-token-type

---

26 **invalid_cursor: a stored cursor replayed after expiry**

The sync job is careful. It follows every cursor, it handles rate limits, and when it is interrupted it writes next_cursor to the database so the next run can pick up where it left off. That last part was tested by restarting the job immediately, which worked perfectly.

`slack_cursor_checkpoint_audit.py`

https://www.allanninal.dev/slack/invalid-cursor/

https://github.com/allanninal/slack-fixes/tree/main/invalid-cursor

---

27 **invalid_limit: asking for more than a page can hold**

Somebody read the pagination documentation, decided that following cursors was a lot of code for a workspace with three hundred channels, and wrote limit=5000. It is an entirely rational thing to try, and Slack answers it with HTTP 200 carrying ok: false and error: invalid_limit.

`slack_page_size_audit.py`

https://www.allanninal.dev/slack/invalid-limit/

https://github.com/allanninal/slack-fixes/tree/main/invalid-limit

---

28 **Membership lost: the bot was removed and nothing said so**

The Monday digest ran for eleven months. Somebody tidied up the channel membership list three weeks ago, the app was in it, and out it went. No email, no error, no ticket. The job still runs, still exits 0, and the only trace is a not_in_channel in a response body nobody reads. The state is easy to check. Nobody checked, because nothing asked them to.

`slack_membership_drift.py`

https://www.allanninal.dev/slack/membership-lost-silently/

https://github.com/allanninal/slack-fixes/tree/main/membership-lost-silently

---

29 **message_limit_exceeded: the workspace posting cap**

Your app posts eleven messages a day. This morning every one of them failed with message_limit_exceeded, and the human-readable half of the error says that members on this team are sending too many messages. Nobody on your team is sending anything. Your retry logic, which is well written and honours Retry-After, does not help, because this is not your quota.

`slack_workspace_send_volume.py`

https://www.allanninal.dev/slack/message-limit-exceeded/

https://github.com/allanninal/slack-fixes/tree/main/message-limit-exceeded

---

30 **Edits, joins and deletes handled as brand new messages**

Somebody fixes a typo in a message they posted an hour ago, and the bot answers it again. Somebody joins the channel, and the ticket pipeline opens a ticket for “Ana has joined the channel”. Somebody deletes a message, and it turns up in the archive anyway, which is the version of this bug that ends up in a compliance conversation.

`slack_message_subtypes.py`

https://www.allanninal.dev/slack/message-subtypes-ignored/

https://github.com/allanninal/slack-fixes/tree/main/message-subtypes-ignored

---

31 **missing_scope tells you the scope needed and the ones you have**

{"ok": false, "error": "missing_scope", "needed": "channels:history", "provided": "chat:write,commands,users:read"}. The developer swears the scope is in the app configuration, and it is — but the app was never reinstalled, so the token in production still carries the grant it was issued with.

`slack_scope_audit.py`

https://www.allanninal.dev/slack/missing-scope-on-read/

https://github.com/allanninal/slack-fixes/tree/main/missing-scope-on-read

---

32 **The app subscribes to no events, so nothing is delivered**

The bot is installed. It is in the channel, which somebody has checked four times. The token is valid, the scopes include app_mentions:read, the request URL is up and returns 200 to a browser. Someone types @deploybot status and nothing happens, and nothing appears in the logs, because nothing arrived.

`slack_event_subscription_audit.py`

https://www.allanninal.dev/slack/no-event-subscriptions/

https://github.com/allanninal/slack-fixes/tree/main/no-event-subscriptions

---

33 **conversations.history clamped to 15 objects and 1 per minute**

A backfill that used to take an hour now takes weeks, and nothing in your code changed. conversations.history still returns ok: true, still returns a valid page, still gives you a cursor — it just returns 15 messages when you asked for 200, and refuses the second call inside the same minute. This is Slack's May 2025 rate-limit change for apps that are not approved for the Marketplace, and it is working exactly as designed.

`slack_history_clamp_probe.py`

https://www.allanninal.dev/slack/non-marketplace-history-clamp/

https://github.com/allanninal/slack-fixes/tree/main/non-marketplace-history-clamp

---

34 **not_allowed_token_type: right secret, wrong token class**

{"ok": false, "error": "not_allowed_token_type"}. The token works. It authenticates, it has scopes, it calls a dozen other methods happily. This one method, and only this one, will not take it — and the error names the problem without naming the solution, because it says what is wrong with the class you brought and nothing about which class it wanted.

`slack_method_token_class.py`

https://www.allanninal.dev/slack/not-allowed-token-type/

https://github.com/allanninal/slack-fixes/tree/main/not-allowed-token-type

---

35 **the token holds admin scopes the app has never called**

The security review asks a reasonable question: why does the nightly digest bot hold admin.users:write, files:write, chat:write.public and users:read.email? Nobody knows. Nothing is broken, no error has ever been logged, and every scope on that list was added by somebody who needed it for ten minutes in 2023.

`slack_scope_surplus.py`

https://www.allanninal.dev/slack/over-broad-scopes/

https://github.com/allanninal/slack-fixes/tree/main/over-broad-scopes

---

36 **next_cursor is ignored so only the first page is ever seen**

The channel inventory has exactly 100 entries. So does the user directory. Nobody questions either number until somebody reports that a channel which plainly exists is missing from the report — and by then the sync has been dropping four fifths of the workspace every night for a year, with ok: true on every single response.

`slack_pagination_audit.py`

https://www.allanninal.dev/slack/pagination-not-followed/

https://github.com/allanninal/slack-fixes/tree/main/pagination-not-followed

---

37 **One quota bucket, eight workers that think they are alone**

The backfill was slow, so it was given eight workers. It got slower. Each worker has its own client, its own connection pool and its own carefully written backoff, and each of them spends most of its life asleep.

`slack_shared_quota_audit.py`

https://www.allanninal.dev/slack/parallel-workers-share-quota/

https://github.com/allanninal/slack-fixes/tree/main/parallel-workers-share-quota

---

38 **chat.postMessage is one per second, per channel**

A fan-out job posts two hundred alerts. Around sixty land. The rest come back ratelimited, and the tier table does not explain it, because the number in the tier table is per minute and this failure happens in the first minute.

`slack_send_cadence.py`

https://www.allanninal.dev/slack/postmessage-one-per-second/

https://github.com/allanninal/slack-fixes/tree/main/postmessage-one-per-second

---

39 **channel_not_found: private, and the token cannot see it**

You are looking at the channel. It is on the screen, the ID is copied from its details panel, other people are talking in it. conversations.info says channel_not_found, and conversations.list returns four hundred channels without it. Nothing is wrong with the ID. The token has not been told that private channels exist.

`slack_private_visibility_probe.py`

https://www.allanninal.dev/slack/private-channel-invisible/

https://github.com/allanninal/slack-fixes/tree/main/private-channel-invisible

---

40 **files made public with a link that works without a Slack login**

Nothing errored, and nothing is going to. Somewhere in the app's history a developer needed an image URL that Block Kit could actually fetch, called files.sharedPublicURL, and it worked. Every file that call has been made against since — customer exports, database dumps, screenshots with a token still on screen — is readable by anyone holding the link, with no Slack account, no workspace membership and no expiry. The flag is on each file, and files.list will hand you all of them.

`slack_public_files_audit.py`

https://www.allanninal.dev/slack/public-file-links-exposed/

https://github.com/allanninal/slack-fixes/tree/main/public-file-links-exposed

---

41 **ratelimited: the Retry-After header nobody read**

The backfill runs beautifully for ninety seconds and then produces a wall of the same line. ratelimited. Sometimes a real HTTP 429, sometimes an HTTP 200 whose body says ok: false. The client sees a failure, does what it does with failures, and tries again immediately, which is the one response guaranteed to make it last longer.

`slack_retry_after_audit.py`

https://www.allanninal.dev/slack/ratelimited-retry-after-ignored/

https://github.com/allanninal/slack-fixes/tree/main/ratelimited-retry-after-ignored

---

42 **restricted_action_read_only_channel: the channel is locked**

The bot is a member: is_member is true and you have checked it twice. The token holds chat:write and the grant screen agrees. The channel is not archived, not private, not the workspace default. The message is still refused, and the error names something that is not in the OAuth screen at all.

`slack_channel_posting_policy.py`

https://www.allanninal.dev/slack/read-only-channel/

https://github.com/allanninal/slack-fixes/tree/main/read-only-channel

---

43 **Slack refresh tokens are single use: a replay kills the pair**

Rotation was working. Then one afternoon every call returns token_expired, the refresh call answers invalid_refresh_token, and the only recovery is to send a customer through the install flow again. Nothing was deployed. What changed is that the service went from one replica to two, and both of them woke up on the same cron minute.

`slack_refresh_ledger_audit.py`

https://www.allanninal.dev/slack/refresh-token-reused/

https://github.com/allanninal/slack-fixes/tree/main/refresh-token-reused

---

44 **The Request URL never echoed back the challenge**

The app is installed. The scopes are right. The bot is in the channel and people are talking to it. Not one line has appeared in the handler's log, ever, because the handler has never been called: on the Event Subscriptions page there is a red line under the Request URL that says the URL did not respond with the value of the challenge parameter.

`slack_request_url_handshake.py`

https://www.allanninal.dev/slack/request-url-unverified/

https://github.com/allanninal/slack-fixes/tree/main/request-url-unverified

---

45 **One missed ack becomes four deliveries and a 429**

It is fine at ten events a minute and it is fine at forty, and then one afternoon it is not fine at anything. The handler slows down a little, some deliveries miss the deadline, Slack sends them again, the extra deliveries make the same API calls, the calls start coming back ratelimited, the handler waits on them, and now it is missing far more deadlines than it was a minute ago.

`slack_retry_amplification.py`

https://www.allanninal.dev/slack/retry-storm-from-event-retries/

https://github.com/allanninal/slack-fixes/tree/main/retry-storm-from-event-retries

---

46 **is_ext_shared: the channel is shared with another org**

Every other note in this batch is about a message that did not arrive. This one is about a message that arrived perfectly, for eight months, in a room that has a vendor in it.

`slack_external_channel_audit.py`

https://www.allanninal.dev/slack/slack-connect-external-channel/

https://github.com/allanninal/slack-fixes/tree/main/slack-connect-external-channel

---

47 **Thread-only and non-threadable channels refuse your post**

This one is not a permission. The app is a member, the token is fine, the channel accepts messages from it all day — just not that message, in that position. A top-level post comes back restricted_action_thread_only_channel. A reply comes back restricted_action_non_threadable_channel, or cannot_reply_to_message, which is a different failure again.

`slack_thread_mode_match.py`

https://www.allanninal.dev/slack/thread-only-or-non-threadable/

https://github.com/allanninal/slack-fixes/tree/main/thread-only-or-non-threadable

---

48 **Three seconds to acknowledge, and what you spend them on**

Somebody submits the modal and gets We had some trouble connecting. Try again? The record was created. It was created twice, actually, and both times correctly. The slash command says operation_timeout and then does the thing anyway.

`slack_ack_budget.py`

https://www.allanninal.dev/slack/three-second-timeout/

https://github.com/allanninal/slack-fixes/tree/main/three-second-timeout

---

49 **A Tier 1 method polled as if it were cheap**

One call throttles. Not the app — one method, over and over, while everything around it is perfectly healthy. Adding backoff makes the loop slower and does not make it work, because the loop is asking for more than the method has ever been allowed to give.

`slack_method_tier_budget.py`

https://www.allanninal.dev/slack/tier1-method-hammered/

https://github.com/allanninal/slack-fixes/tree/main/tier1-method-hammered

---

50 **token_expired every 12 hours because rotation is on**

The app is perfect for half a day after every deploy and then every call returns {"ok": false, "error": "token_expired"}. A nightly redeploy hid it for four months, because each deploy ran the install flow again and each install handed back a fresh token. Then the redeploy was removed as an optimisation, and the app started dying every lunchtime.

`slack_rotation_clock.py`

https://www.allanninal.dev/slack/token-expired-rotation/

https://github.com/allanninal/slack-fixes/tree/main/token-expired-rotation

---

51 **token_revoked: the app is gone and retrying will not help**

One tenant of your multi-workspace app has received nothing for six weeks. No alert fired. The installation row is still there, still marked active, still being handed to the scheduler every fifteen minutes, and every call it makes returns {"ok": false, "error": "token_revoked"}. Somebody removed the app from Manage apps in January and your store has been retrying ever since.

`slack_dead_install_sweep.py`

https://www.allanninal.dev/slack/token-revoked/

https://github.com/allanninal/slack-fixes/tree/main/token-revoked

---

52 **every Slack profile has a null email and nothing errored**

The nightly user sync has been green for four months. It reads every member out of Slack, writes them to the warehouse, and joins them against the HR system on email. The join has matched nothing since the day it shipped, because every row it wrote has email = null — and users.list returned ok: true, with complete-looking profiles, every single night.

`slack_email_scope_audit.py`

https://www.allanninal.dev/slack/users-read-email-missing/

https://github.com/allanninal/slack-fixes/tree/main/users-read-email-missing

---